

tier1_container_e2e

tier1_container_e2e.sh — Tier-1 emulated-device END-TO-END test on podman

Last verified: 2026-06-10 (run-id 20260610T105150Z, podman 5.8.2, arm64 Linux guest)

Overview

tests/emulator/tier1_container_e2e.sh proves Tier-1 emulated-device testing works **end-to-end against a containerized Helix OTA control plane**, with no live hardware. It boots the control plane (ota-server) and the ota-device-emulator (the containers/ submodule runtime image, per §11.4.76) in a shared podman pod, waits for the control plane to report healthy, runs the **real register → update-check → telemetry round-trip** via the emulator container, asserts the round-trip from captured podman logs, and tears the stack down. “We boot, we run, test and validate.”

Anti-bluff (§11.4): every PASS is backed by real captured podman output written under docs/qa/<run-id>/. If a step fails for an environment reason it reports the real error and exits non-zero — it never fakes green.

Prerequisites

- **podman** with a running machine (on macOS: podman machine start). The Linux guest is **arm64**, so binaries cross-compile GOOS=linux GOARCH=arm64.
- **Go 1.26** toolchain on the host. The server module (server/go.mod) has replace directives pointing at sibling paths OUTSIDE server/ (../containers, ../submodules/http3, ../submodules/ota-protocol, the §11.4.28 co-developed bricks), so the binaries are built **on the host** where those resolve — not inside a container.

Usage

```
bash tests/emulator/tier1_container_e2e.sh
KEEP_UP=1 bash tests/emulator/tier1_container_e2e.sh
# leave the stack up to inspect
```

Optional environment overrides: PODMAN, CP_IMAGE, EMU_IMAGE, ADMIN_USER, ADMIN_PASS, HELIX_PORT.

What it does (6 phases)

1. **(a) cross-compile** bin/ota-server + bin/ota-device-emu as static linux/arm64 binaries on the host.
2. **(b) build images** with podman: the control-plane image (server/Dockerfile), the base ota-device-emu image (the unchanged submodule containers/images/ota-device-emu/Dockerfile), and a **derived** emulator image that COPYs our binary into the submodule base.
3. **(c) boot** a podman pod (helix-tier1-e2e) with the control-plane + (later) emulator containers sharing one network namespace — the emulator reaches the control plane on 127.0.0.1 (the on-demand-infra invariant; the boot IS the test entry point, no manual podman ... up).
4. **(d) wait** for control-plane /healthz to return {"status":"ok"}.
5. **(e) run** the emulator container for one register → update-check → telemetry cycle (–once), capturing its JSON Outcome.
6. **(f) assert** the round-trip from captured evidence: a server-assigned device_id (REGISTER), on_target=true + the explicit 204 note (UPDATE-CHECK), and healthy=true (telemetry/health). Then teardown.

Evidence

Written under docs/qa/<run-id>/ (run-id = UTC timestamp, §11.4.83):

- tier1_container_e2e_transcript.txt — full phase-by-phase transcript.
- emulator_outcome.json — the emulator's structured Outcome (the device-side round-trip proof).
- control_plane.log / control_plane_after_cycle.log — control-plane boot + post-cycle logs.
- emulator_stderr.log — emulator diagnostics (empty on success).

Why COPY, not volume-mount

On a macOS podman machine, /Users is mounted into the Linux VM but the repo's /Volumes/... path is **not** — a runtime -v of the host binary fails (statfs ...: no such file or directory). The harness therefore stages each binary into a build context (server/.docker-bin/, tests/emulator/.docker-bin/ — both gitignored, §11.4.30) and COPYs it into the image at build time, which podman streams from the context regardless of the VM mount table. Portable everywhere.

Edge cases / honest boundaries

- **Clean on-target 204.** With the in-memory store and no deployment staged, the cycle is register (201) → update-check (204 on-target) → JSON Outcome. No update is offered, so no apply/telemetry batch runs (applied=false); the on-target 204 IS the honest round-trip the harness asserts (its contract's "clean on-target 204" branch). A staged-deployment variant would additionally exercise the full download→installing→installed→verifying→success telemetry lifecycle.
- **Quiet server log.** ota-server runs in gin ReleaseMode, so the control-plane log shows boot lines but not per-request lines; the device-side Outcome (a server-assigned device_id could only exist if the server handled login + register, and a 204 only if it handled the update poll) is the authoritative server-handling proof.
- **HEALTHCHECK directive.** podman's default OCI image format ignores HEALTHCHECK (it warns and is valid only for docker-format images / the docker runtime). The harness performs the authoritative health check itself by probing / healthz over HTTP.

Related scripts / files

- server/Dockerfile — control-plane runtime image.
- containers/images/ota-device-emu/Dockerfile — emulator runtime base (submodule).
- server/cmd/ota-server/main.go, server/cmd/ota-device-emu/main.go — binaries.
- server/internal/config/config.go — the HELIX_* env interface.